Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Фундаментальные науки»

КАФЕДРА «Прикладная математика»

Лабораторная работа № 2
по дисциплине «Типы и структуры данных»

Тема Сравнительный анализ рекурсивного и нерекурсивного алгоритмов

Студент Михайлов А.К.

Группа ФН12-31Б

Преподаватели Волкова Л.Л.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

1.1 Нерекурсивный алгоритм 5

1.2 Рекурсивный алгоритм 5

2 Конструкторская часть **7**

2.1 Нерекурсивный алгоритм вычисления суммы цифр числа 7

2.2 Ререкурсивный алгоритм вычисления суммы цифр числа 8

3 Технологическая часть **9**

3.1 Реализация и выбор средств 9

3.2 Тестирование программы 9

4 Исследовательская часть **11**

4.1 Сравнение времени выполнения программы 11

4.2 Оценка трудоемкости алгоритмов 11

4.3 Оценка затрат памяти 13

ЗАКЛЮЧЕНИЕ **14**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **15**

ПРИЛОЖЕНИЕ А **16**

ВВЕДЕНИЕ

В данной лабораторной работе рассматриваются два алгоритма (**рекурсивный и нерекурсивный**), позволяющие посчитать сумму цифр натурального числа).

Целью выполнения является реализация указанных алгоритмов, а также проведение их сравнительного анализа по скоростным характеристикам и эффективности. Для достижения поставленных задач необходимо выполнить следующие пункты:

1) разработать рекурсивный и нерекурсивный алгоритмы решения задачи, согласно варианту;

2) описать средства разработки и инструменты замера процессорного времени выполнения реализации алгоритмов;

3) реализовать разработанные алгоритмы на языке программирования C++;

4) выполнить тестирование реализации алгоритмов;

5) выполнить теоретическую оценку затрачиваемой реализацией каждого алгоритма памяти;

6) выполнить замеры процессорного времени выполнения реализации алгоритмов в зависимости от варьируемого входа;

7) сделать выводы из сравнительного анализа реализации рекурсивного и нерекурсивного алгоритмов решения одной и той же задачи, заданной вариантом, по критериям: емкостной эффективности и временной эффективности.

1 Аналитическая часть

Рекурсивный и **нерекурсивный** алгоритмы представляют собой различные подходы к решению одной и той же задачи. В одном случае решение достигается за счет последовательных вызовов функции самой себе, а в другом — с помощью итеративных структур, таких как циклы. Оба метода позволяют реализовать решение, однако отличаются по способу организации вычислений и управлению ресурсами.

1.1 Нерекурсивный алгоритм

Нерекурсивный алгоритм — это алгоритм, основанный на использовании циклов (for $while$ и $do-while$) и последовательного повторения набора инструкций без вложенных вызовов функций [1]. Такой подход обычно более эффективен по расходу памяти и предотвращает риск переполнения стека при обработке больших объемов данных.

Нерекурсивный алгоритм вычисления суммы цифр числа заключается в использовании цикла, в котором поэлементно извлекаются цифры числа, начиная с последней, и эти цифры последовательно суммируются. Процесс продолжается до тех пор, пока число не станет равным нулю. В ходе выполнения цикла извлекается последняя цифра числа с помощью операции взятия по модулю ($n \bmod 10$), эта цифра добавляется к аккумулируемой сумме, а число делится на 10 (с округлением вниз), чтобы избавиться от уже учтённой последней цифры. В конце, после завершения цикла, возвращается полученная сумма. Этот цикл выполняется, пока $n \neq 0$, и по завершении возвращается сумма всех извлечённых цифр.

1.2 Рекурсивный алгоритм

Рекурсивный алгоритм — это алгоритм, который решает задачу, вызывая сам себя для обработки меньших частей этой задачи. Каждый вызов функции решает небольшую часть проблемы, а итоговое решение достигается объединением результатов всех вызовов [1].

Рекурсивный алгоритм строится на следующих принципах:

— **базовый случай**: условие, при котором рекурсия останавливается и возвращается готовый ответ. Обычно это простая ситуация, которую проще решить напрямую.

— **рекурсивный шаг**: функция вызывает сама себя с меньшей или упрощённой версией исходных данных, приближаясь к базовому случаю при каждом вызове.

Рекурсивный алгоритм вычисления суммы цифр числа основывается на разбиении числа n на последнюю цифру и оставшуюся часть числа. Формула для суммы цифр в рекурсивной реализации задаётся следующим образом:

$$\text{sumDigits}(n) = \begin{cases} 0, & \text{если } n = 0 \\ (n \bmod 10) + \text{sumDigits}(\frac{n}{10}), & \text{если } n > 0 \end{cases} \tag{1.1}$$

где sumDigits(n) — рекурсивная функция вычисления суммы цифр числа n. Основанием рекурсии является случай, когда $n = 0$, и в этом случае результат равен 0. В общем случае алгоритм извлекает последнюю цифру через операцию взятия по модулю ($n \bmod 10$), а оставшуюся часть числа получает целочисленным делением на 10 ($\frac{n}{10}$). Эти операции повторяются рекурсивно, пока n не станет равен нулю.

Вывод

Были описаны рекурсивный и нерекурсивный алгоритмы вычисления суммы цифр числа.

2 Конструкторская часть

2.1 Нерекурсивный алгоритм вычисления суммы цифр числа

Схема нерекурсивного алгоритма вычисления суммы цифр числа представлена на рисунке 2.1.

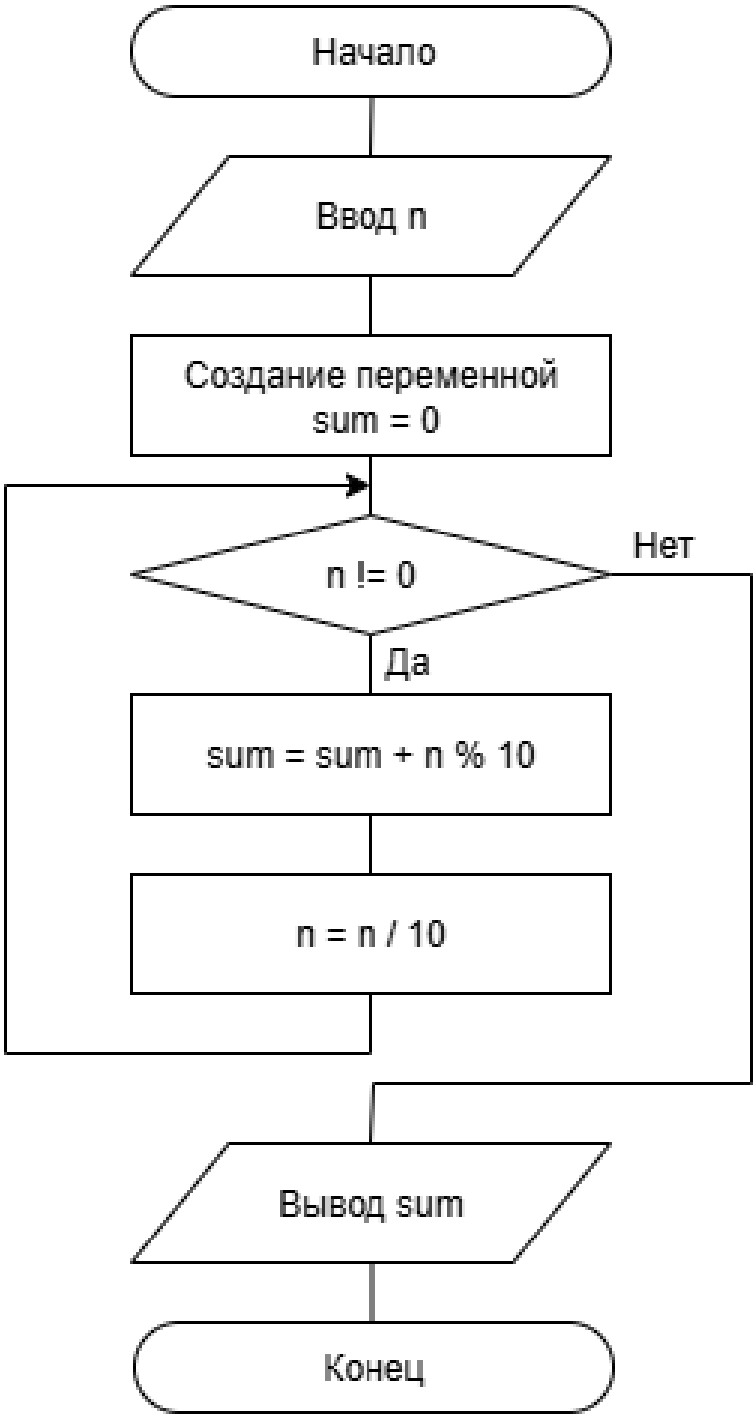

Рисунок 2.1 — Блок-схема **нерекурсивного алгоритма**

2.2 Реекурсивный алгоритм вычисления суммы цифр числа

Схема рекурсивного алгоритма вычисления суммы цифр числа представлена на рисунке 2.2.

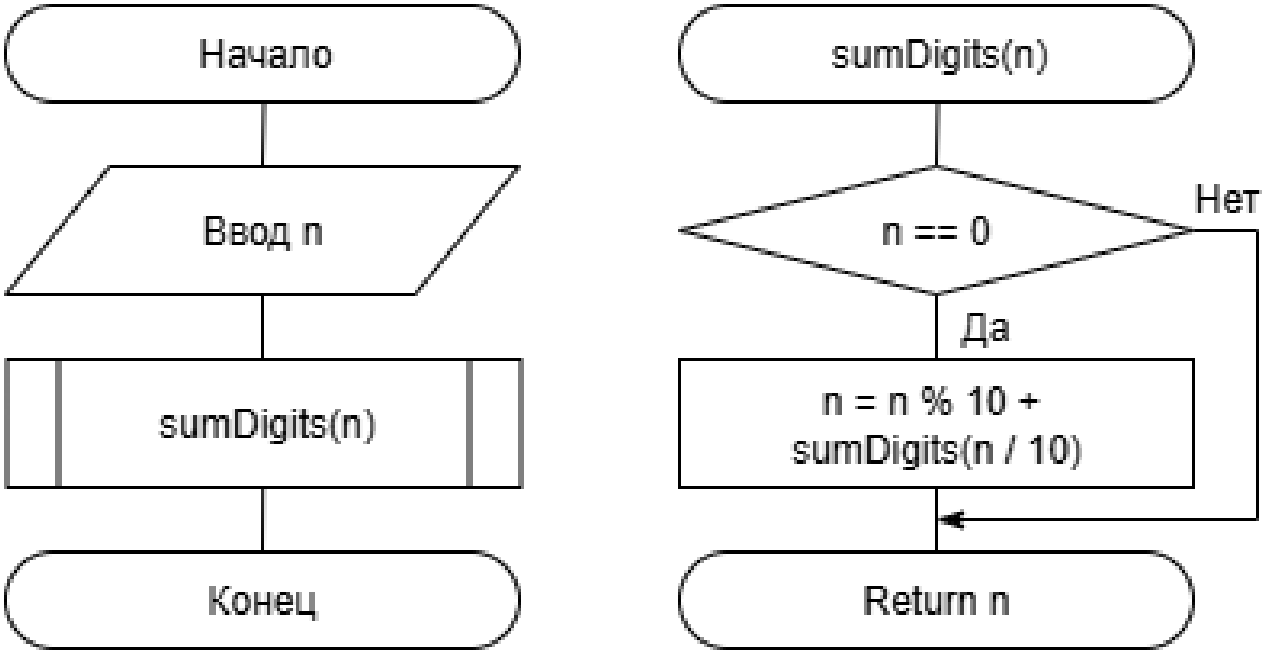

Рисунок 2.2 — Блок-схема **рекурсивного алгоритма**

Вывод

Была выполнена задача разработки основных алгоритмов.

3 Технологическая часть

3.1 Реализация и выбор средств

Для программной реализации алгоритмов использовалась среда разработки $Visual\ Studio\ Code$ [2]. Язык программирования, на котором была выполнена реализации алгоритмов — $C++$.

В листинге А.1 представлена программная реализация описанных алгоритмов.

3.2 Тестирование программы

Ниже в таблице 3.1 приведено тестирование программы и её проверка на ожидаемый/получаемый результат.

Таблица 3.1 — Результаты тестирования

Описание тестирования	**Входные данные**	**Ожидаемый результат**	**Полученный результат**
Проверка на ввод нечисловых значений	AAA	Это не число. Попробуйте снова	Это не число. Попробуйте снова
Проверка входных данных на превышение диапазона int	1111111 1111111	Введите 1 для выбора пользовательского режима Введите 2 для выбора режима массированного замера Введите натуральное целое число Число вне диапазона int или некорректное. Попробуйте снова	Введите 1 для выбора пользовательского режима Введите 2 для выбора режима массированного замера Введите натуральное целое число Число вне диапазона int или некорректное. Попробуйте снова
Проверка на корректность работы программы	123456	Введите 1 для выбора пользовательского режима Введите 2 для выбора режима массированного замера Введите натуральное целое число Сумма цифр числа с использованием рекурсии: 21 Сумма цифр числа без использования рекурсии: 21	Введите 1 для выбора пользовательского режима Введите 2 для выбора режима массированного замера Введите натуральное целое число Сумма цифр числа с использованием рекурсии: 21 Сумма цифр числа без использования рекурсии: 21

Вывод

Было проведено тестирование программы на работоспсобность.

4 Исследовательская часть

4.1 Сравнение времени выполнения программы

Исследование по измерению времени работы реализаций алгоритмов проводилось на ноутбуке Huawei Matebook d16 [3]. Характеристики ноутбука: 64-разрядная операционная система Windows 11 [4], 8-ядерный процессор Intel Core i5-12450H [5] x64 с тактовой частотой 2 Ггц, оперативная память DDR4 8 ГБ. Для компиляции кода использовался компилятор g++.

На рисунке 4.1 представлены графики зависимости процессорного времени выполнений реализаций всех вышеописанных алгоритмов вычисления количества цифр в числе (**рекурсивного** и **нерекурсивного**) от размера умножаемых матриц. На вход реализации алгоритмов 10000 раз получали n-значное число, $n \in [1; 10]$, с шагом, равным 1. Числа генерировались случайным образом в формате int. Использованные данные для графиков — это результат измерения с помощью функции $modeTest$ в программе.

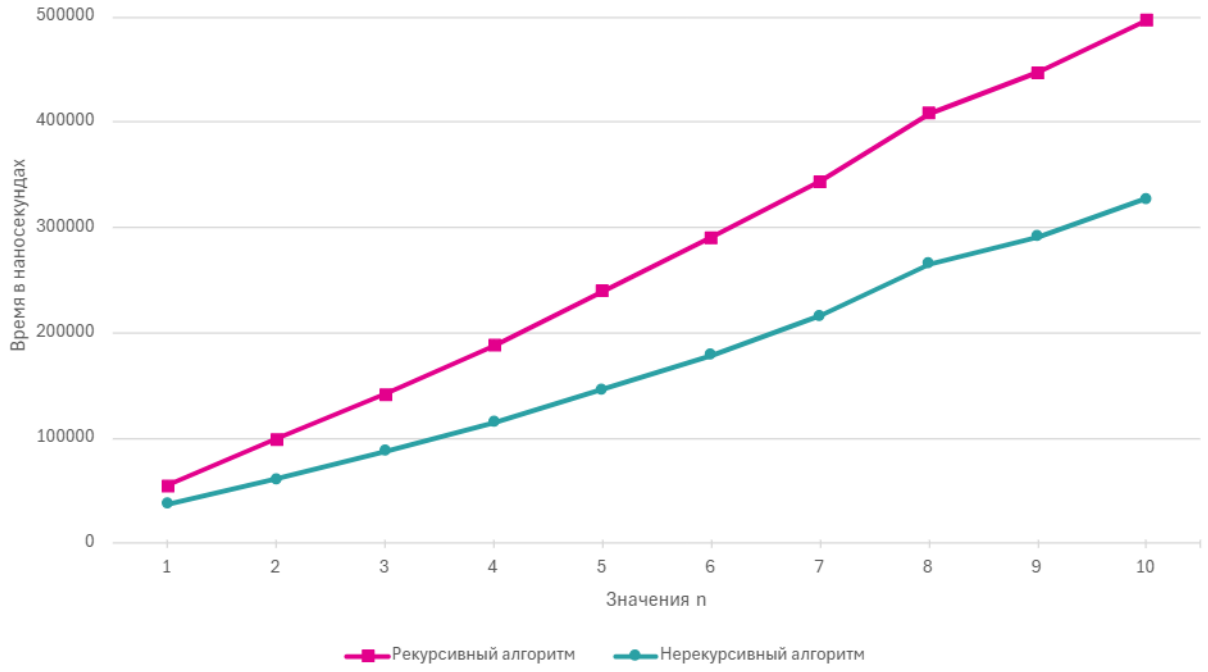

Рисунок 4.1 — Графики зависимости времени работы реализаций алгоритмов от размероа исходного числа

4.2 Оценка трудоемкости алгоритмов

Ниже приведен теоретический расчёт сложности всех алгоритмов. Модель трудоемкости введена в таблице 4.1.

Таблица 4.1 — Модель трудоемкости

Название базовой операции	**Стоимость**
Сложение	1
Вычитание	1
Присвивание	1
Равно	1
Меньше	1
Больше	1
Меньше или равно	1
Больше или равно	1
Не равно	1
Инкремент	1
Декремент	1
Плюс с присваиванием	1
Индексирование	1
Умножение	2
Деление	2
Целочисленное деление	2
Остаток от деления	2

Оценка трудоемкости условного оператора:

$$f = f_{\text{усл}} + \left[\begin{array}{l} \left[\begin{array}{l} \min(f_A, f_B), \text{ лучший случай,} \\ \max(f_A, f_B), \text{ худший случай,} \end{array} \right. \quad f_A \neq f_B \\ f_A, \quad \text{при } f_A = f_B \end{array} \right. \tag{4.1}$$

где f_A, f_B – стоимости блоков кода в заивимости от истинности условия в блоке ЕСЛИ. Если услвоие истинно, выбирается минимум из двух приведенных, в противном случае выбирается максимум,

$f_{\text{усл}}$ — стоимость блока условия.

Оценка трудоемкости цикла $while$:

$$f = 2 \cdot f_{\text{усл}} + M \cdot (f_{\text{тел}} + f_{\text{усл}}), \tag{4.2}$$

где $f_{\text{усл}}$ — стоимость условия, накладываемого на переменную,

$f_{\text{тел}}$ — стоимость тела цикла,

M — количество итераций цикла.

Пусть $T(n)$ — трудоемкость алгоритма для входного числа n.

Нерекурсивный алгоритм (1.1): Пусть N — количество цифр в числе n.

$$T(n) = 1 + 2 + N \cdot (1 + 2 + 2) = 3 + 5 \cdot N, \tag{4.3}$$

где N — количество цифр числа n.

Рекурсивный алгоритм (1.2): Для n = 0 алгоритм будет иметь трудоемкость, равную 1. Если n будет больше 0, то алгоритм будет иметь следующую трудоемкость:

$$T(n) = 1 + 5 + T\left(\frac{n}{10}\right) = 6 + T\left(\frac{n}{10}\right) \tag{4.4}$$

4.3 Оценка затрат памяти

В данном разделе будет проведена оценка затраченной двумя алгоритмами памяти.

Нерекурсивный алгоритм

В данном алгоритме для хранения данных используется только одна переменная типа int. Сложность по памяти такого алгоритма составляет $O(1)$.

Рекурсивный алгоритм

Для оценки памяти рекурсивного алгоритма необходимо использовать высоту дерева рекурсивных вызовов. При каждом вызове функции происходит:

— создание локальной переменной и, что занимает $O(1)$ памяти;

— выполнение рекурсивного вызова с аргументом, равным целой части деления n на 10.

Поскольку на каждом шаге число n делится на 10 и рекурсия продолжается до тех пор, пока n не станет равным 0, глубина рекурсии равна количеству цифр числа n, что примерно равно $[\log_{10} n] + 1$. Следовательно, высота дерева рекурсивных вызовов равна $O(\log n)$.

Память, необходимая для каждого вызова, является $O(1)$, а суммарная память — это сумма памяти всех вызовов по глубине рекурсии:

$$O(\log n) \cdot O(1) = O(\log n) \tag{4.5}$$

Таким образом, общая оценка памяти рекурсивной функции по сравнению с входными данными составляет $O(\log n)$.

Вывод

На основе проведенного анализа времени выполнения и использования памяти было установлено, что нерекурсивный алгоритм является предпочтительным из двух реализованных вариантов для решения поставленной задачи, поскольку работает быстрее при обработке одинаковых данных и обладает меньшей затратой памяти по сравнению с рекурсивным.

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы была достигнута цель работы: реализация алгоритмов, а также сравнительный анализ алгоритмов умножения матриц.

Также были выполнены все задачи:

1) разработаны рекурсивный и нерекурсивный алгоритмы решения задачи согласно варианту;

2) были описаны средства разработки и инструменты для замера процессорного времени выполнения реализованных алгоритмов;

3) реализованы разработанные алгоритмы на языке программирования $C++$;

4) проведено тестирование реализованных алгоритмов;

5) введена теоретическая оценка используемой памяти каждым алгоритмом;

6) выполнены замеры процессорного времени выполнения алгоритмов в зависимости от варианта входных данных;

7) сделаны выводы из сравнительного анализа реализации рекурсивного и нерекурсивного алгоритмов по критериям: емкостной эффективности и временной эффективности.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы современных алгоритмов 2-е дополненное издание / Дж. Макконелл // под ред. С.Кландо, доп. М.В.Ульянова. — Москва: ТЕХНОСФЕРА, 2004. — 370 с.

2. Visual Studio Code documentation. [Электронный ресурс]. URL: `https://code.visualstudio.com/docs` (дата обращения: 11 октября 2025).

3. Huawei. [Электронный ресурс]. URL: `https://consumer.huawei.com/ru/laptops/matebook-14-2024/specs/?ysclid=mg5k19xzko358223160` (дата обращения: 11 октября 2025).

4. Microsoft Ignite. [Электронный ресурс]. URL: `https://learn.microsoft.com/ru-ru/windows` (дата обращения: 11 октября 2025).

5. NanoReview. [Электронный ресурс]. URL: `https://nanoreview.net/ru/cpu/intel-core-i5-12450h` (дата обращения: 11 октября 2025).

ПРИЛОЖЕНИЕ А

В листинге А.1 представлена программная реализация описанных алгоритмов.

```
#include <iostream>
#include <chrono>
#include <string>
#include <random>

using namespace std;
using namespace chrono;

struct Timer {

  Timer() {
    start = high_resolution_clock::now();
    end = high_resolution_clock::now();
  }

  void start_time() { start = high_resolution_clock::now(); }
  void stop_time() { end = high_resolution_clock::now(); }

  void get_time(string method_name) {
    auto duration = duration_cast<nanoseconds>(end - start);
    cout << "Время выполнения " << method_name << " алгоритма составило " <<
   duration.count() << " наносекунд" << endl;
  }

  private:
  time_point<high_resolution_clock> start;
  time_point<high_resolution_clock> end;
};

int get_random_value(int digits_count) {

  int min = 1;
  for (int i = 1; i < digits_count; ++i)
  min *= 10;
  int max = min * 10 - 1;

  static random_device rd;
  static mt19937 gen(rd());

  uniform_int_distribution<int> dis(min, max);
  return dis(gen);

}

```

```
bool isNumber(const string& s) {

  int start = 0;

  if (s.empty())
  return false;

  for (int i = start; i < s.size(); i++) {
    if (!isdigit(s[i]))
    return false;
  }

  return true;

}

bool isInt(const string& s) {

  try {
    size_t pos;
    long long temp = stoll(s, &pos);
    if (pos != s.size())
    return false;
    if (temp < INT_MIN || temp > INT_MAX)
    return false;
    return true;
  }
  catch (const out_of_range&) { return false; }
  catch (const invalid_argument&) { return false; }

}

int stringToInt(const string& s) {
  return stoi(s);
}

int safeInputInt() {

  string input;
  while (true) {

    cin >> input;

    if (!isNumber(input)) {
      cout << "Это не число. Попробуйте снова" << endl;
      continue;
    }
```

```

    if (!isInt(input)) {
      cout << "Число вне диапазона int или некорректное. Попробуйте снова" << endl;
      continue;
    }

    return stringToInt(input);
  }
}

int sum_digits_recursion(int n) {
  if (n == 0)
  return 0;
  return (n % 10) + sum_digits_recursion(n / 10);
}

int sum_digits_non_recursion(int n) {
  int sum = 0;
  while(n != 0) {
    sum += n % 10;
    n /= 10;
  }
  return sum;
}

void mode_user() {

  cout << "Введите натуральное целое число" << endl;
  int num = safeInputInt();
  cout << "Сумма цифр числа с использованием рекурсии: " << sum_digits_recursion(
   num) << endl;
  cout << "Сумма цифр числа без использования рекурсии: " <<
   sum_digits_non_recursion(num) << endl;

}

void mode_test() {

  int num, temp;
  for (int i = 1; i <= 10; i++) {

    cout << "Замер для " << i << "значных- чисел:" << endl;
    num = get_random_value(i);
    Timer time1, time2;

    time1.start_time();
    for (int k = 0; k < 10000; k++)
```

```
    temp = sum_digits_recursion(num);
    time1.stop_time();
    time1.get_time("рекурсивного  ");

    time2.start_time();
    for (int k = 0; k < 10000; k++)
    temp = sum_digits_non_recursion(num);
    time2.stop_time();
    time2.get_time("нерекурсивного");

  }

}

int main() {

  int menu;
  cout << "Меню:"
  << endl << "Введите 1 для выбора пользовательского режима"
  << endl << "Введите 2 для выбора режима массированного замера" << endl;

  do {
    menu = safeInputInt();
    if (menu != 1 && menu != 2) { cout << "Введите 1 или 2" << endl; }
  } while (menu != 1 && menu != 2);

  if (menu == 1)
  mode_user();
  else
  mode_test();

  return 0;

}
```

Листинг А.1 — Программная реализация разных алгоритмов умножения матриц